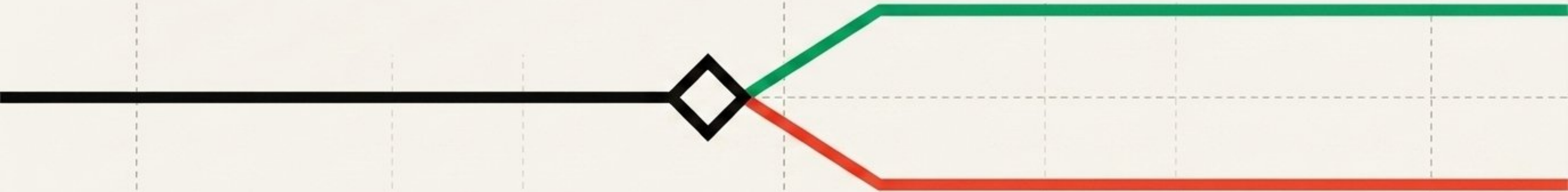


Detection of Toxic Language & Threats



Evaluating Computational Trade-offs: Base BERT+LIME vs. Sentiment-Routed Architectures

Istanbul Medipol University
Algorithm Analysis Course Project



The Solved Problem

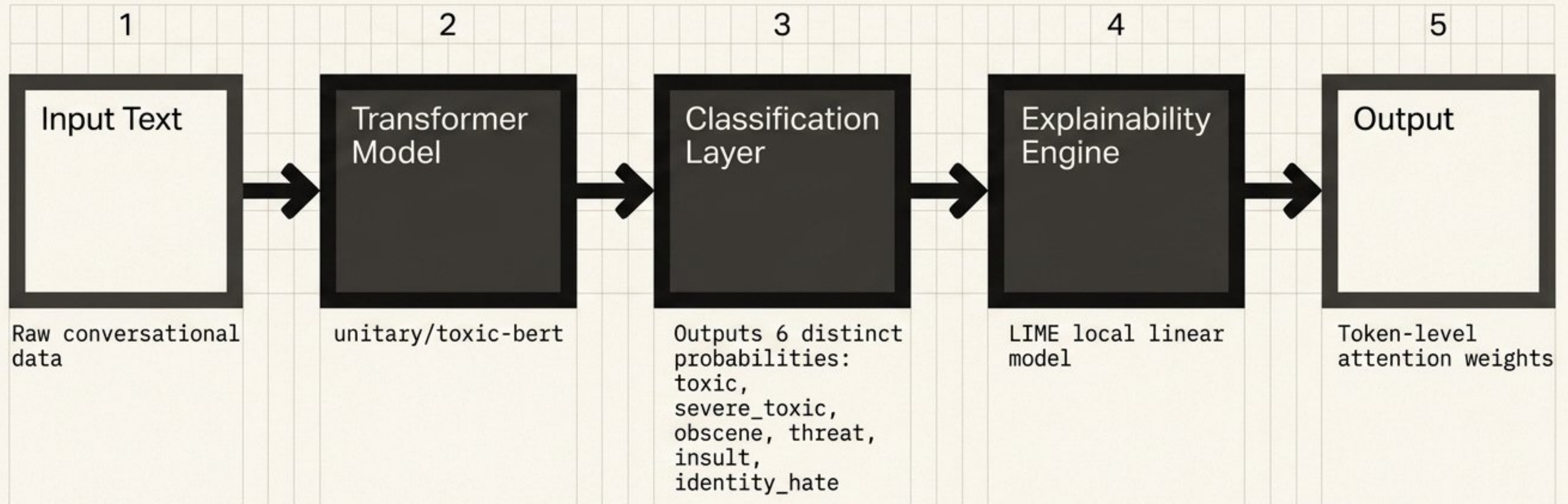
Public projects achieve 98%+ accuracy on toxicity detection. Building another standard classifier adds zero value.

“ A moderator cannot act on on a number alone.”

The Real Challenge: High accuracy is useless without transparent, explainable predictions.

01. Was a comment flagged for a minor insult or a serious threat?
02. Does the user receive a warning, a ban, or get ignored?
03. We must know exactly which words triggered the flag.

Approach 1: The Base Case Pipeline



Key Mechanism: Every single piece of input text must traverse this entire heavy deep-learning pipeline to extract exact word-level weights.

Explainability in Action: Token-Level Weights

THE HEAVY TRACK: HIGH TOXICITY DETECTED

Input: i hate you noob

Prediction: TOXIC (97.63%)

Granular Breakdown:

insult: 87.73%  87.73%
obscene: 56.18%  56.18%
severe_toxic: 2.44%  2.44%
threat: 0.26%
identity_hate: 6.27%  6.27%

LIME Attention Map:

'noob' (+0.465) ↑ 'hate' (+0.156) ↑

THE BENIGN TRACK: SAFE CONTENT

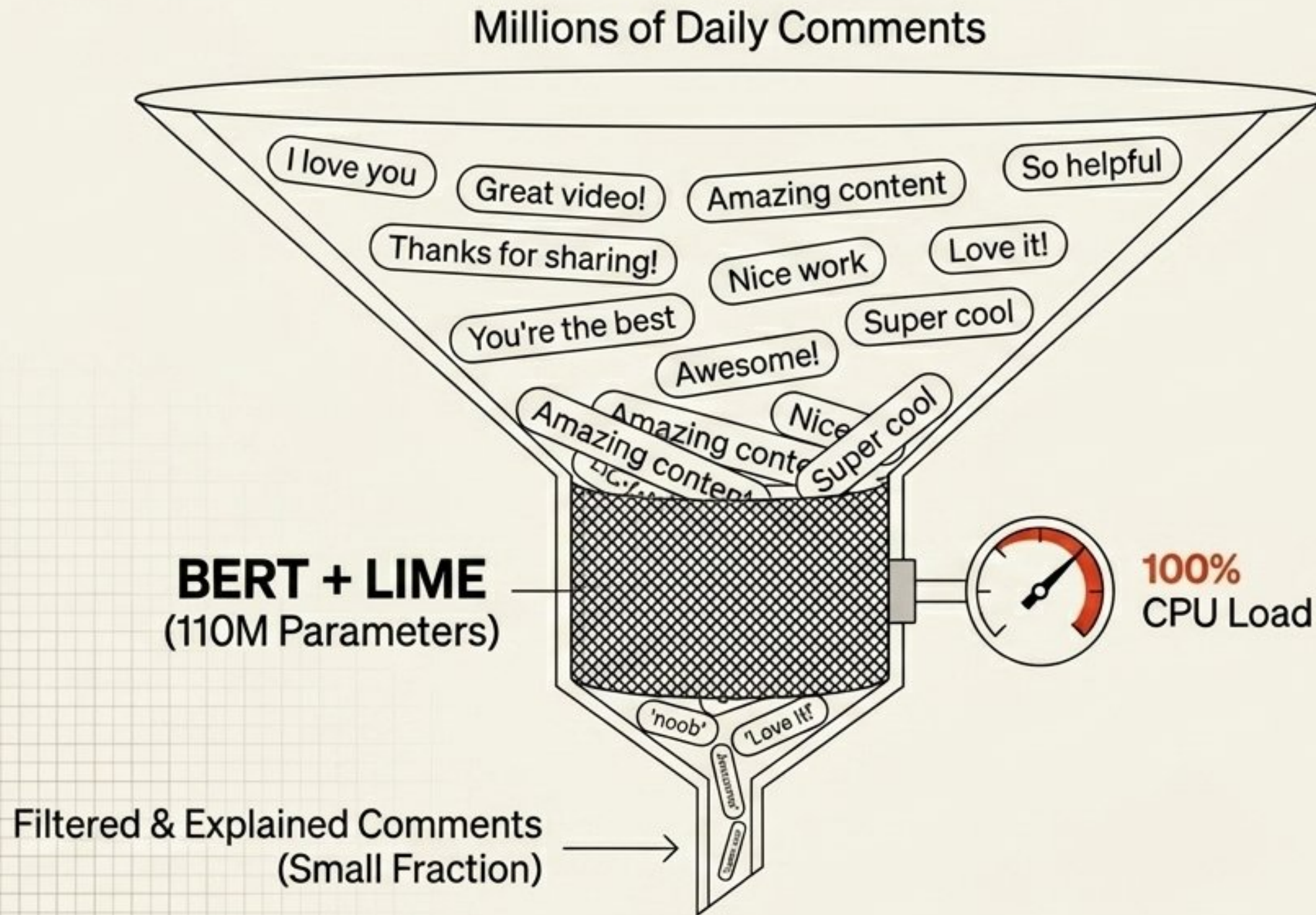
Input: i love you

Prediction: TOXIC (0.09%)

Action: Safely Ignored 

Takeaway: The model successfully highlights the exact trigger words, giving moderators precise context.

The Monolithic Bottleneck



The Scale Problem

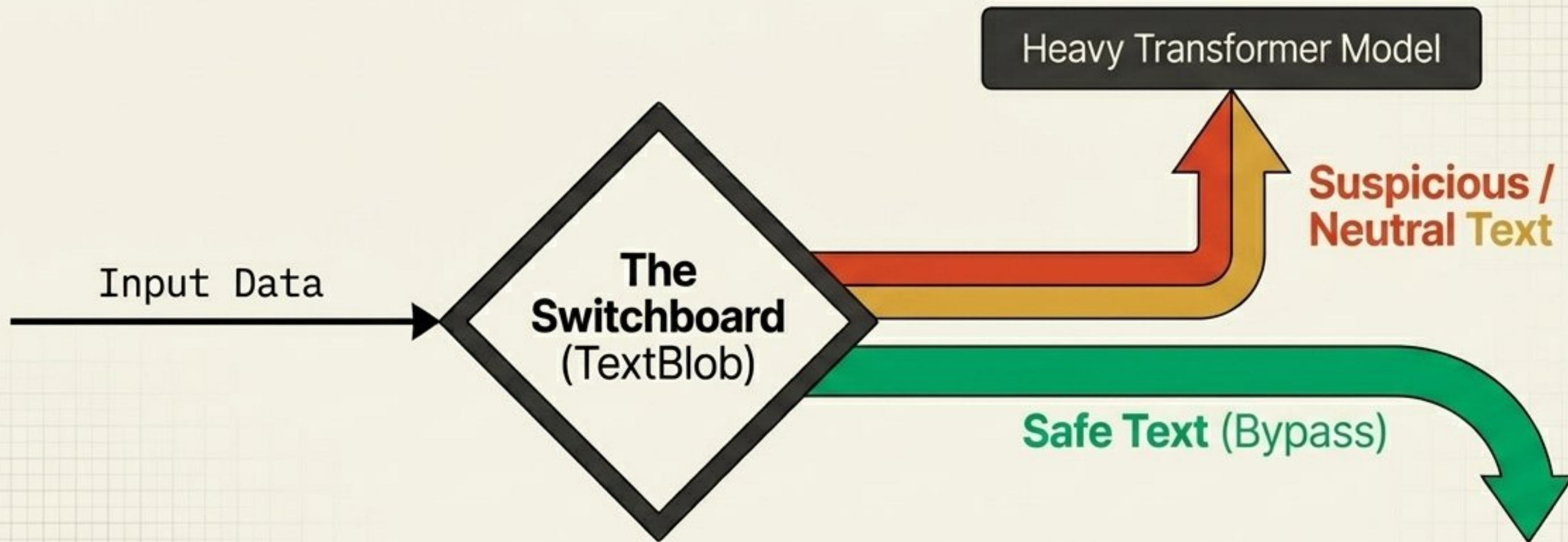
Social platforms process millions of comments per second.

The Inefficiency

Running a deep transformer model AND complex LIME perturbation calculations on every single comment is computationally **wasteful**.

Why are we spending GPU/CPU cycles analysing the exact token weights of benign text?

Approach 2: The Optimised Routing Architecture



The Hypothesis

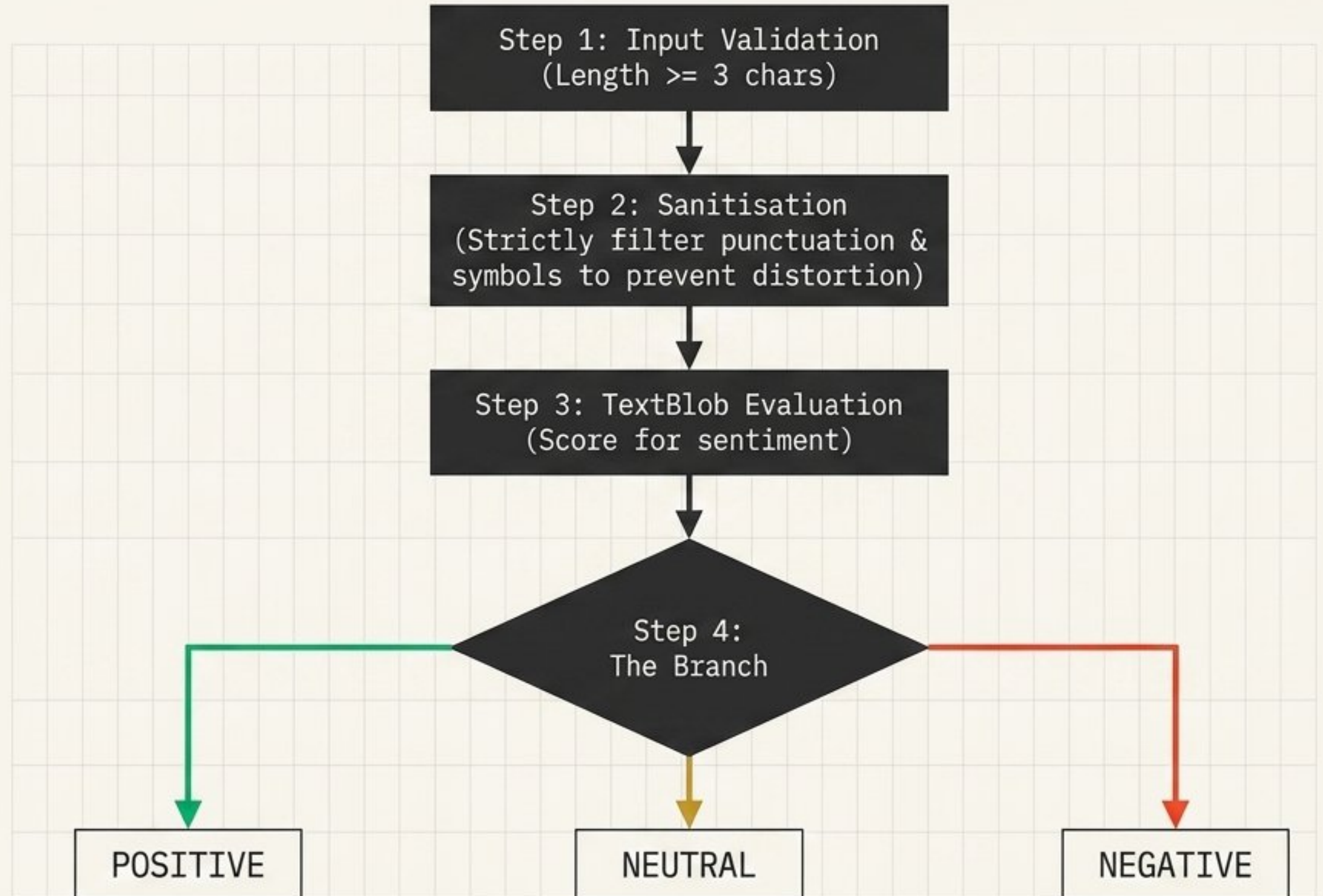
We can safely bypass heavy computation for clearly safe text without compromising overall platform safety.

The Solution

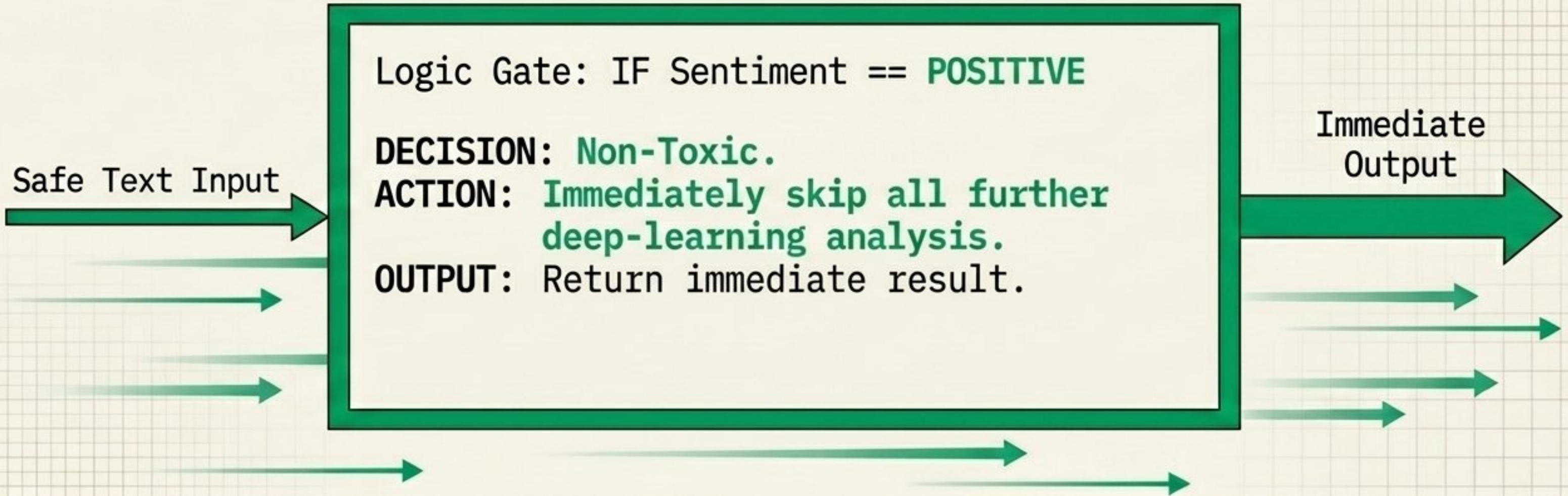
Deploy a lightweight, deterministic initial layer. By using TextBlob for rapid sentiment analysis, we create an algorithmic gatekeeper.

Pre-Processing & The Sentiment Switchboard

The architectural split relies on stripping away conversational noise before initial evaluation, ensuring clean routing.

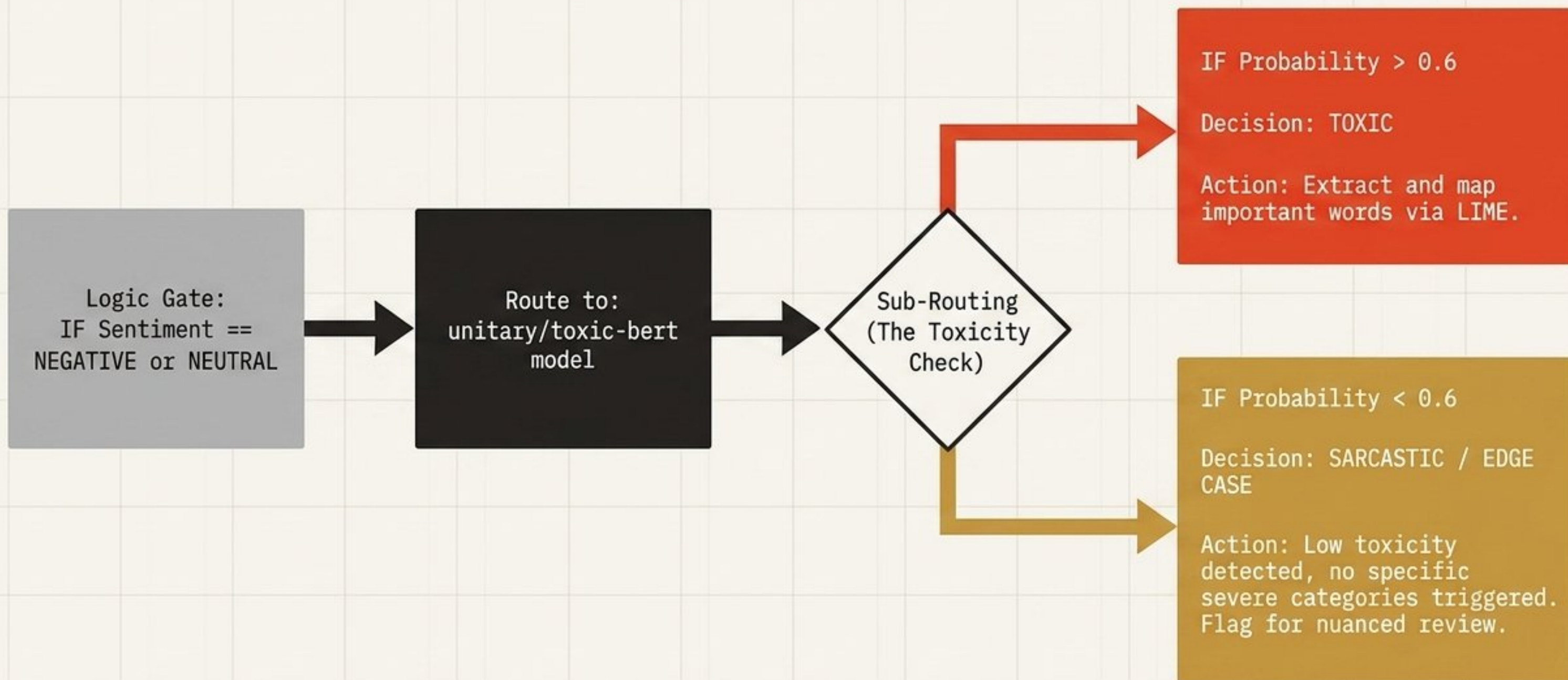


The Fast Track: Bypassing Computation



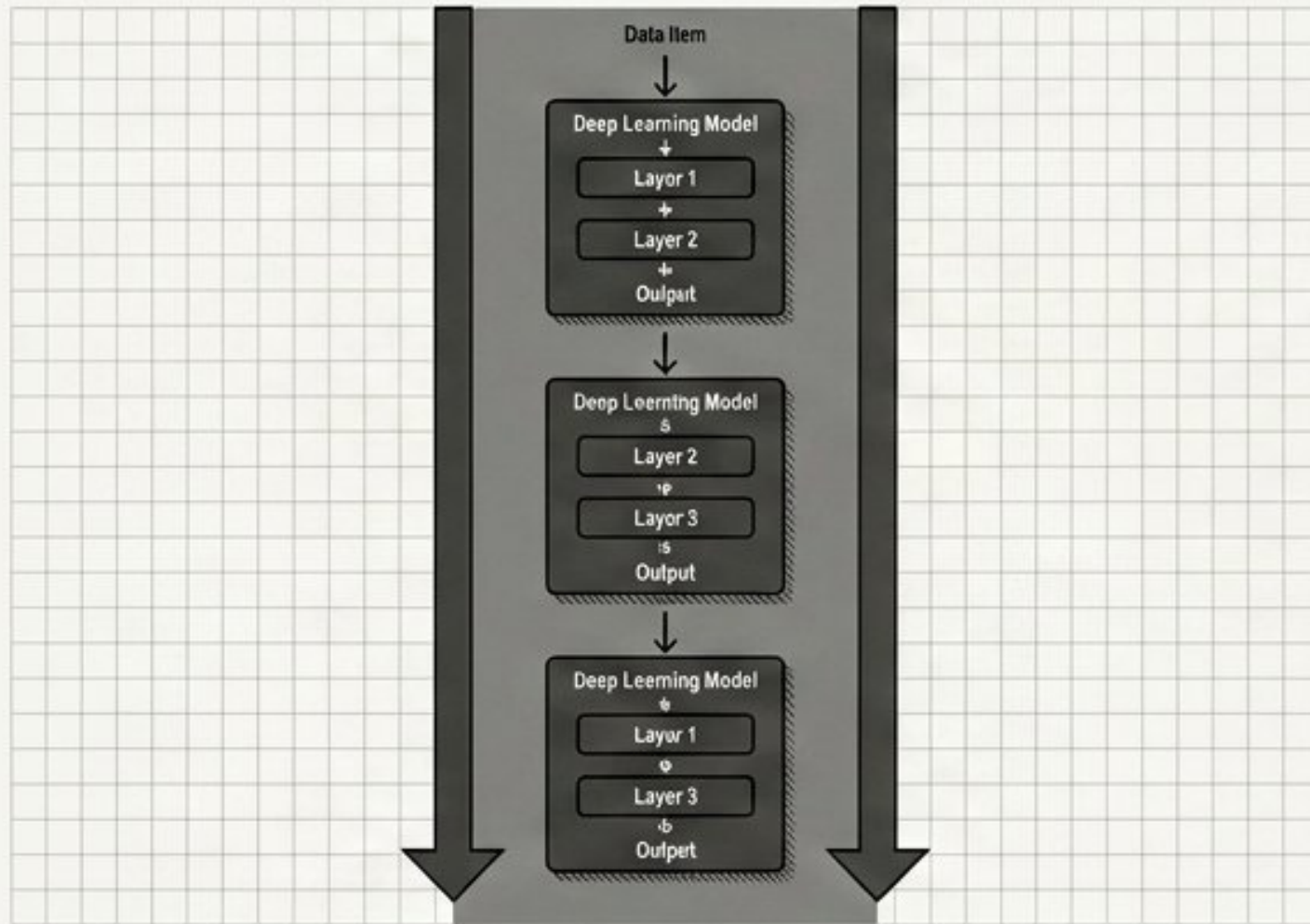
The Engineering Impact: By circumventing the heavy transformer model entirely, this logic reduces computational load and latency for this specific request to near-zero milliseconds.

The Heavy Track: Evaluating the Grey Areas



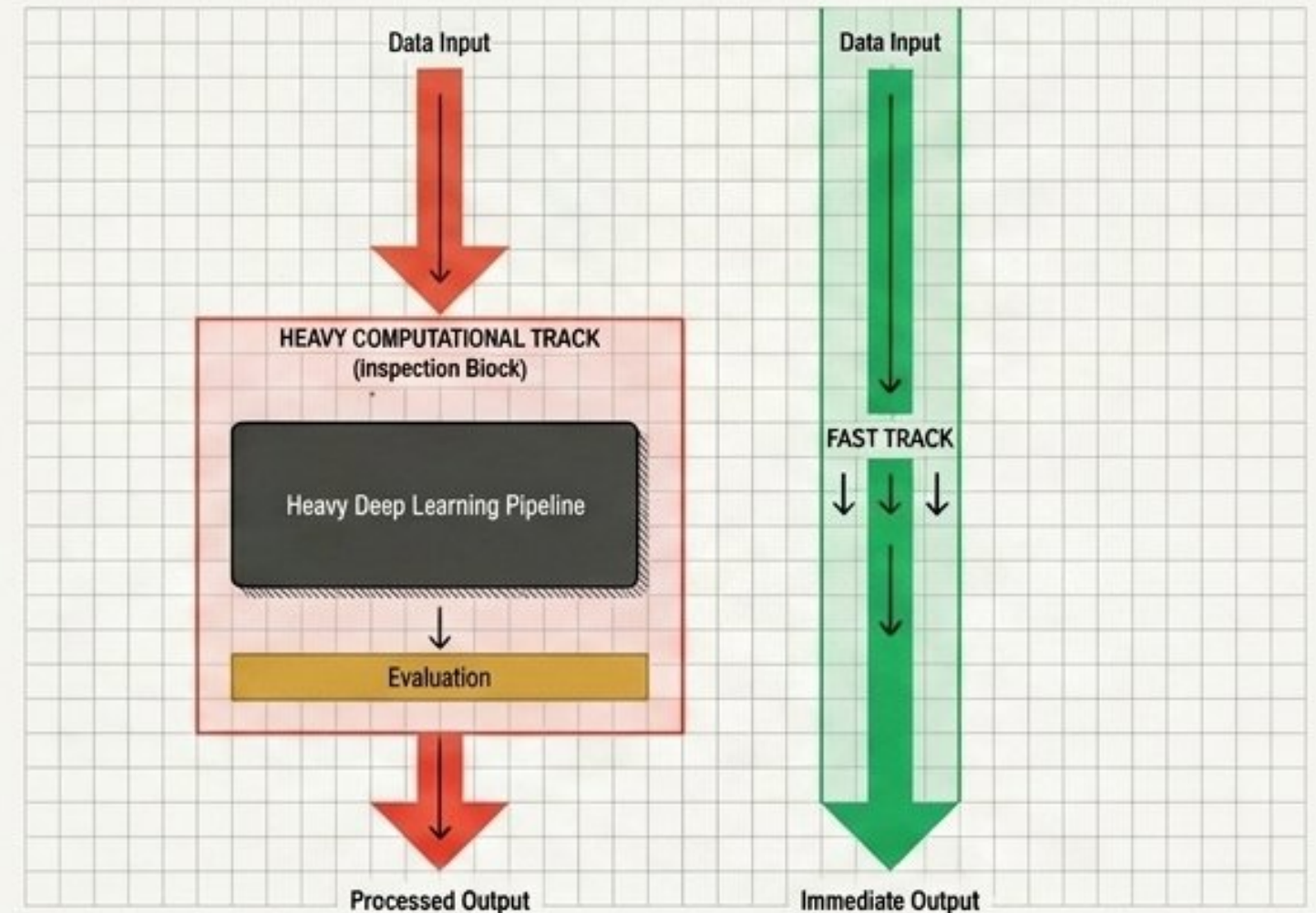
Structural Comparison: Monolithic vs. Micro-Routed

Base Case



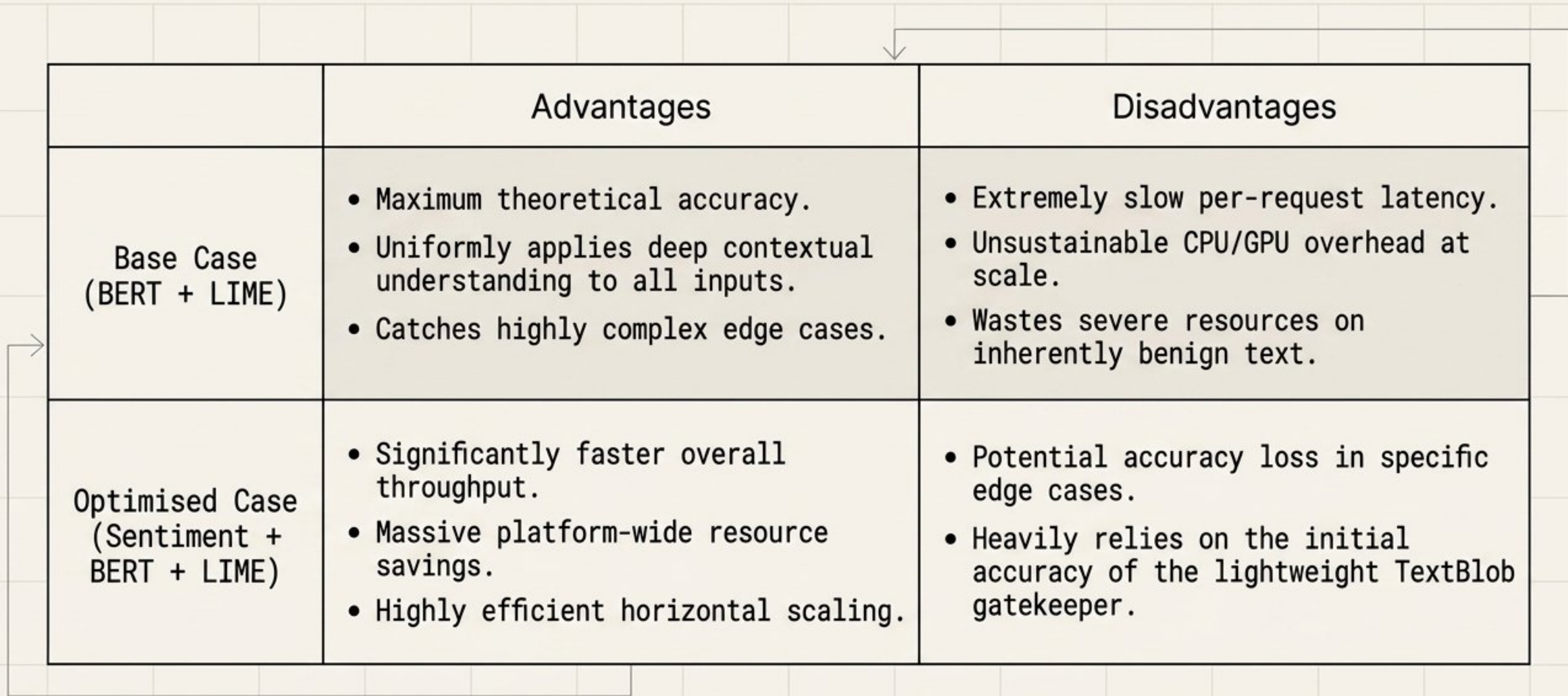
Uniform processing logic. Every item, regardless of complexity, incurs maximum latency moving through the exact same deep-learning pipeline.

Optimised Case



Dynamic routing. System throughput is no longer static; it is determined by the ratio of positive to negative comments in the real-world dataset.

The Trade-Off Matrix: Speed vs. Precision



	Advantages	Disadvantages
Base Case (BERT + LIME)	• Maximum theoretical accuracy. • Uniformly applies deep contextual understanding to all inputs. • Catches highly complex edge cases.	• Extremely slow per-request latency. • Unsustainable CPU/GPU overhead at scale. • Wastes severe resources on inherently benign text.
Optimised Case (Sentiment + BERT + LIME)	• Significantly faster overall throughput. • Massive platform-wide resource savings. • Highly efficient horizontal scaling.	• Potential accuracy loss in specific edge cases. • Heavily relies on the initial accuracy of the lightweight TextBlob gatekeeper.

Analysing the Edge Cases: Slang & Disguised Toxicity

You guys are killing it! Absolutely sick!

The Dilemma

Human context recognizes this as positive slang.

Because the Optimised Route registers a high positive sentiment score, it bypasses the BERT model entirely.

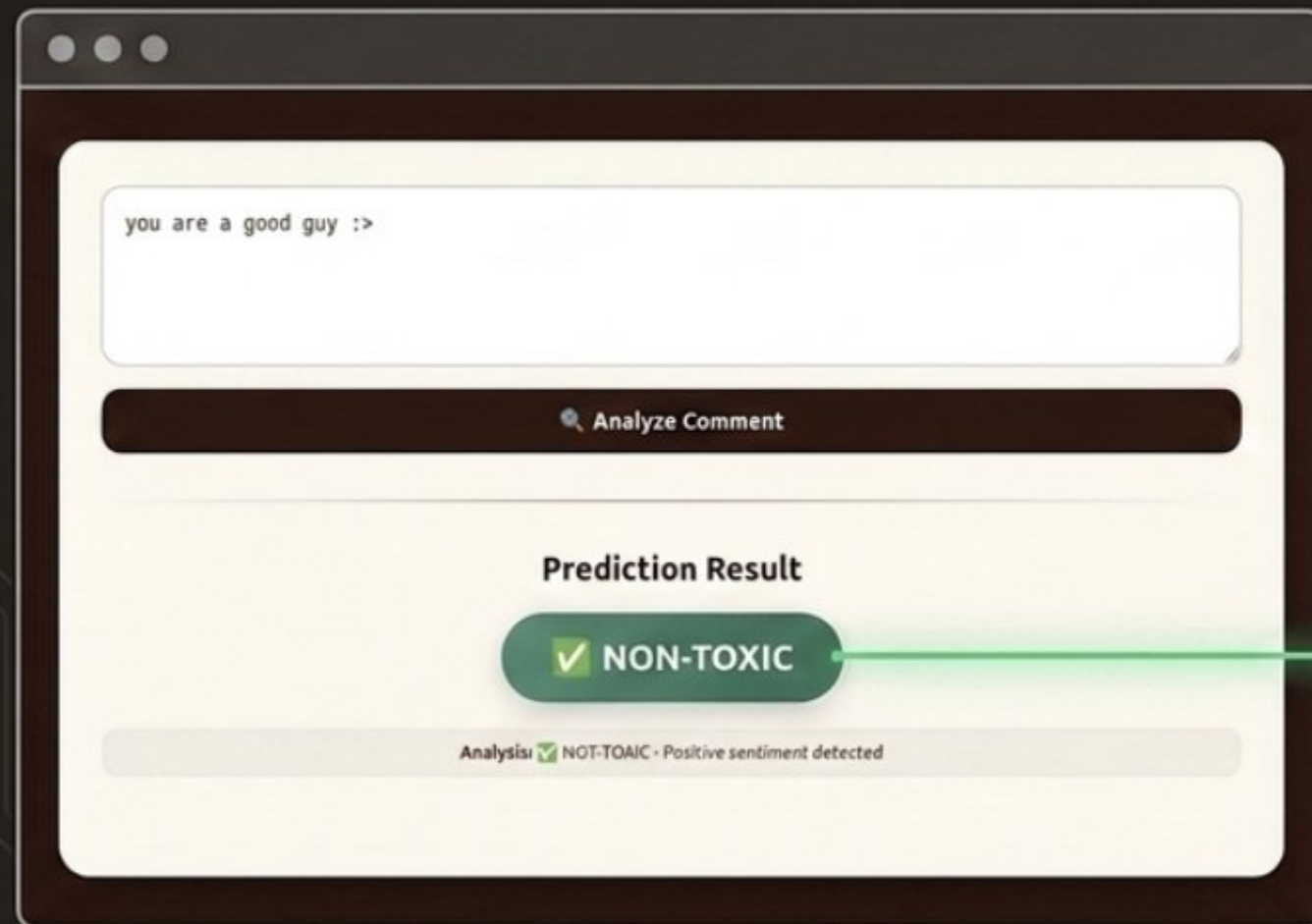
The Risk: What if the comment is toxic sarcasm disguised with positive framing?

The Calculated Trade-off

By routing positive sentiment away from BERT, we willingly sacrifice the detection of ultra-niche, positively-framed toxicity. In exchange, we gain massive system-wide speed and architectural stability.

Flask Application: Final Results & Deployment

System Validation 01: The Computational Bypass



The screenshot shows a web application interface with a dark theme. At the top, there's a text input field containing the comment "you are a good guy :>". Below the input field is a button labeled "Analyze Comment". Underneath the button, the "Prediction Result" is displayed as a green pill-shaped button with a checkmark and the text "NON-TOXIC". At the bottom, a status bar shows "Analysis: NOT-TOXIC - Positive sentiment detected". A green line connects the "NON-TOXIC" result to the "0(1) FAST PATH ENGAGED" box on the right.

0(1) FAST PATH ENGAGED

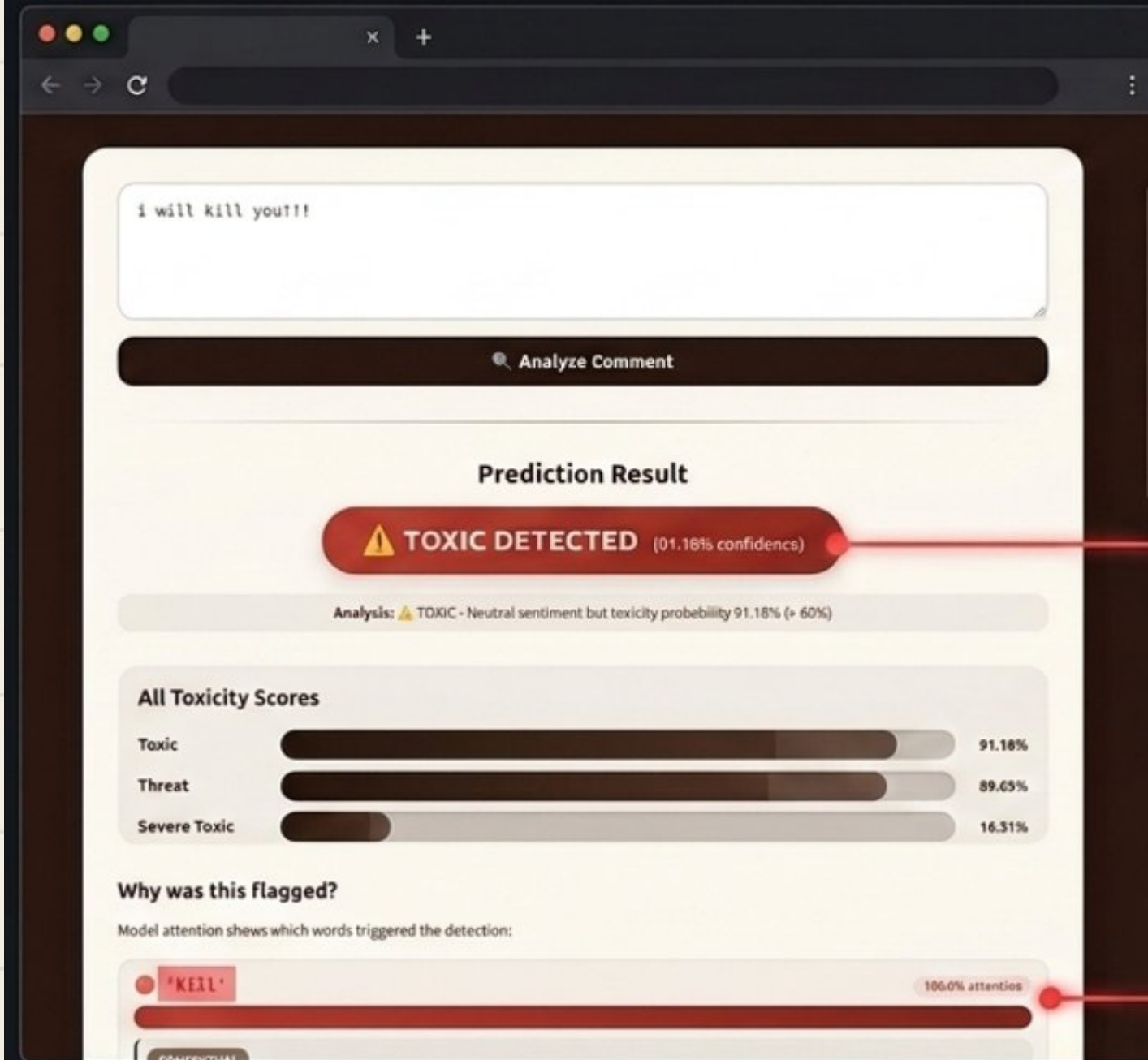
TextBlob registers positive sentiment. The system intentionally skips the BERT implementation. All probability matrices are forced to 0.

RESOURCE SAVINGS

By routing this comment away from the transformer model, the system saves approximately ~85% of the computational load for this specific request.

Flask Application: Final Results & Deployment

System Validation 02: High-Threat Routing & Explainability



HEAVY COMPUTE ENGAGED

Neutral/Negative initial sentiment triggers the full BERT model.
Probability threshold exceeds 0.6 (91.18% confidence).

LIME EXPLAINABILITY

Token-level attention weights isolate the word 'kill' (100% attention), fulfilling the core project objective of algorithmic transparency for moderators.

Flask Application: Final Results & Deployment

System Validation 03: Edge-Case Handling (Sarcasm)

Toxic Language Detector

You have less map awareness than Christopher Columbus

Analyze Comment

Prediction Result

SARCASTIC DETECTED

Analysis: SARCASTIC - Negative sentiment but low toxicity probability 0.58% (= 60%)

THRESHOLD LOGIC

TextBlob identifies negative sentiment, triggering BERT. However, BERT's primary toxicity probability returns at 0.58%.

EDGE-CASE CLASSIFICATION

The intersection of negative sentiment and < 0.6 toxicity probability triggers the custom 'SARCASTIC' classification. This prevents false positives while still flagging mildly antagonistic banter.

Project Roadmap & Deployment Status



01 | Core Engine

BERT + LIME
integration

[STATUS: DONE]

02 | Mapping

Exact token-to-word
categorisation
filtering

[STATUS: DONE]

03 | Interface

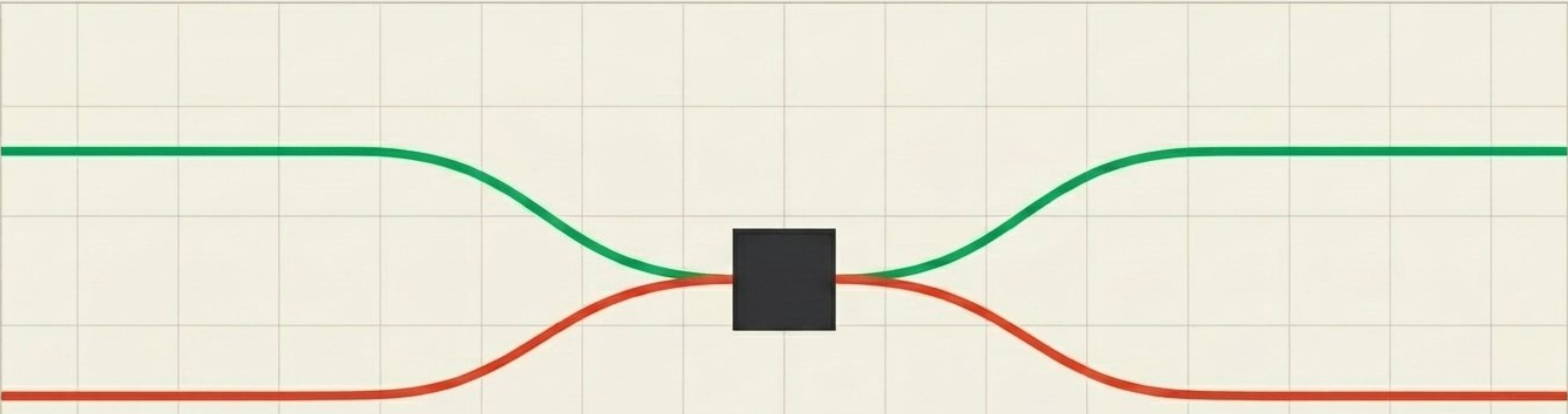
Interactive Flask
web application
deployment

[STATUS: DONE]

04 | Finalisation

Technical
documentation &
Final report

[STATUS: DONE]



True Algorithmic Mastery

Building a safe platform is no longer just about achieving 98% accuracy in a vacuum. True algorithmic design is the ability to deploy that heavy accuracy exactly when it is needed, and the intelligence to bypass it when it is not.